

Estructuras de datos del núcleo asociadas a los procesos

El núcleo mantiene diferentes estructuras de datos asociadas a los procesos, las cuales son imprescindibles para la ejecución de los mismos. Algunas de estas estructuras como la *pila del núcleo*, el *área U* y la *tabla de regiones por proceso* son locales a cada proceso, es decir, cada proceso tiene asignada su propia estructura privada. Otras estructuras, como la *tabla de procesos* y la *tabla de regiones* son globales para todos los procesos, es decir, sólo existe en el núcleo una estructura para todos los procesos.

Por otra parte, algunas de estas estructuras como la *pila del núcleo* y el *área U* se implementan en el espacio de direcciones virtuales de cada proceso. Mientras que otras como la *tabla de procesos*, la *tabla de regiones por proceso* y la *tabla de regiones* se implementan en el espacio de direcciones virtuales del núcleo. Todas estas estructuras tienen en común que son propiedad del núcleo y por tanto solo pueden ser accedidas en modo núcleo.

Otras estructuras de datos del núcleo asociadas a los procesos son las *tablas de páginas* y la *tabla de descriptores de ficheros*. Las tablas de páginas permiten traducir las direcciones de memoria virtual a direcciones de memoria física, (se estudiarán en el Tema 9). Por su parte, la *tabla de descriptores de ficheros* identifica a los ficheros abiertos por un proceso.

Pila del núcleo

La existencia en UNIX de dos modos distintos de ejecución, modo usuario y modo núcleo, hace necesario la existencia de una pila independiente para cada modo y para cada proceso: la *pila de usuario* y la *pila del núcleo*. La *pila de usuario* contiene los marcos de pila de las funciones que se ejecutan en modo usuario. De forma análoga, la *pila del núcleo* contiene los marcos de pila de las funciones que se ejecutan en modo núcleo. Por tanto, estas dos pilas crecerán de forma autónoma. De hecho la pila del núcleo está vacía cuando el proceso se ejecuta en modo usuario.

Por otra parte, puesto que pueden estar ejecutándose en paralelo varias instancias de una misma rutina del núcleo asociada cada una de ellas a un proceso distinto, se hace necesario la existencia de una pila del núcleo distinta para cada proceso. Por tanto, la *pila del núcleo* se puede definir como una estructura local a cada proceso que contiene los marcos de pila de las funciones o rutinas del núcleo invocadas durante la ejecución del proceso en modo núcleo. Frecuentemente la pila del núcleo se implementa dentro del *área U* del proceso, pero también puede implementarse en un *área de memoria independiente*.

Ejemplo:

Considérese el programa siguiente escrito en lenguaje C, el cual crea una copia del contenido de un fichero en otro fichero nuevo. Supóngase que el nombre del fichero ejecutable que se crea después de compilar este programa es **copfile**. Un usuario invocaría a este programa desde un terminal (\$) escribiendo:

```
$ copfile file_V file_N
```

donde **file_V** es el nombre de un fichero ya existente en el sistema que se desea copiar y **file_N** es el nombre del nuevo fichero.

El sistema invoca **[2]** a la función principal `main` suministrándole el número de parámetros `argc` en la lista `argv`, e inicializando a cada miembro del array `argv` para que apunte a un parámetro suministrado por el usuario. En la forma de invocación de este programa el número de parámetros es 3, `argv[0]` apunta al string de caracteres `copfile` (el nombre del programa es usualmente usado como parámetro 0), `argv[1]` apunta al array de caracteres `file_V` y `argv[2]` apunta al array de caracteres `file_N`.

```

[1] char buffer[2048];
[2] main(int argc, char *argv[]){
[3]     int aviejo, anuevo;
[4]     if (argc!=3){
[5]         printf("Error, usar 2 parametros");
[6]         exit(1);
[7]     }
[8]     aviejo=open(argv[1],0444);
[9]     if (aviejo == -1){
[10]        printf("Error: No se puede abrir el archivo %s\n", argv[1]);
[11]        exit(1);
[12]    }
[13]    anuevo=creat(argv[2],0666);
[14]    if (anuevo == -1) {
[15]        printf("Error: No se puede crear el archivo %s\n", argv[2]);
[16]        exit(1);
[17]    }
[18]    copiar(aviejo,anuevo);
[19]    exit(0);
[20] }

[21] copiar (int viejo, int nuevo){
[22]     int contador;
[23]     while ((contador = read(viejo,buffer,sizeof(buffer))) > 0)
[24]         write(nuevo,buffer,contador);
[25] }

```

En primer lugar [4] comprueba si ha sido invocado con un número erróneo de parámetros, es decir, si argc es distinto de 3. En caso afirmativo, imprime [5] en pantalla el mensaje

Error, usar 2 parámetros

e invoca [6] a la llamada al sistema exit para terminar la ejecución del programa. En caso negativo, invoca [7] a la llamada al sistema open para que abra con permisos de sólo lectura para todos los usuarios (máscara de modo octal 0444) el fichero file_V. Esta llamada devuelve un descriptor del fichero que es almacenado en la variable aviejo.

Si la llamada al sistema open falla [8], es decir, aviejo=-1, entonces imprime [9] en pantalla el mensaje

Error: No se puede abrir el archivo file_V

e invoca [10] a la llamada al sistema exit para terminar la ejecución del programa. En caso contrario, invoca [11] a la llamada al sistema creat para crear el fichero file_N. con permisos de lectura y escritura para todos los usuarios (máscara de modo octal 0666) el fichero file_N. Esta llamada devuelve un *descriptor del fichero* y lo asocia a la variable anuevo.

Si la llamada al sistema falla [12], es decir, anuevo=-1, entonces imprime [13] en pantalla el mensaje

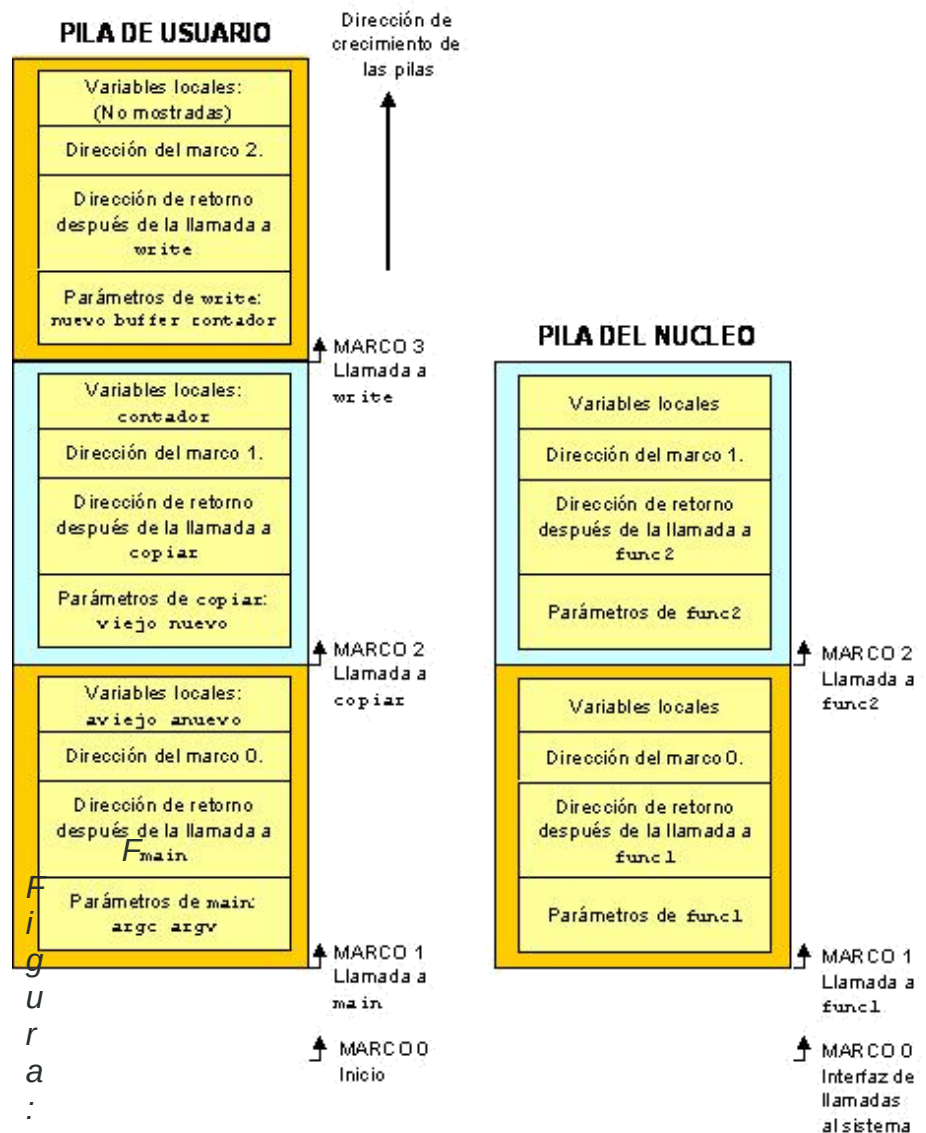
Error: No se puede crear el archivo file_N

e invoca [14] a la llamada al sistema exit para terminar la ejecución del programa. En caso contrario el programa llama [15] a la función copiar, la cual entra en un lazo [19], donde se invoca a la llamada al sistema read que lee un total de 2048 bytes procedentes del fichero file_V y los almacena en la zona de memoria asignada [1] al array de caracteres buffer.

A continuación invoca [20] a la llamada al sistema write para escribir los datos en el nuevo fichero. La llamada al sistema read devuelve el número de bytes leídos y los almacena en la variable contador, devolviendo 0 cuando llega al final del fichero. El programa finaliza el lazo cuando encuentra el final del fichero o cuando hay algún error durante la llamada al sistema, es decir, read devuelve el valor -1 (obsérvese que el programa no comprueba la aparición de errores durante la llamada al sistema write). Entonces vuelve de la función copiar e invoca [16] a la llamada al sistema exit para terminar la ejecución del programa.

En la figura se muestra un posible esquema de la *pila de usuario* y de la *pila del núcleo* en un cierto instante de la ejecución en un sistema UNIX del proceso asociado al archivo ejecutable copfile. En concreto, las pilas representadas corresponden al instante en que se está ejecutando la llamada al sistema write (sentencia [20]). Se observa que la *pila de usuario* contiene tres marcos de pila: el primero asociado a la función main, el segundo a la función copiar (invocada dentro de la función main) y el tercero a la llamada al sistema write (invocada dentro de la función copiar). El marco 0 es un marco mudo que el núcleo utiliza para inicializar la pila.

Es importante resaltar que un proceso invoca a una llamada al sistema como si se tratase de una función cualquiera, lo que supone que se creará en su pila de usuario un marco de pila para dicha función. Si bien en una pila de usuario nunca podrán existir simultáneamente dos marcos de pila asociados a dos llamadas al sistema ya que hasta que no se invoque y se vuelva de la primera llamada no se podrá invocar a la segunda.



Pila del usuario y pila del núcleo en un cierto instante de la ejecución de la llamada al sistema write

De hecho, una pila de usuario en la que exista un marco de pila para una llamada al sistema no podrá crecer hasta que no se vuelva de dicha llamada al sistema.

Un proceso que invoca a una llamada al sistema en realidad está invocando a la función de librería asociada a dicha llamada al sistema, que entre sus diferentes instrucciones posee una interrupción software o trap que produce la conmutación hardware al modo núcleo. Se comienza a ejecutar código del núcleo, por lo que se utilizará la pila del núcleo asociada a dicho proceso.

Por ejemplo, supóngase que en un cierto instante de la ejecución de la llamada al sistema write se requiere ejecutar una cierta función del núcleo func1, y que ésta a su vez invoca a otra cierta función del núcleo func2. En consecuencia la *pila del núcleo* contendrá dos marcos de pila: el primero asociado a la función del núcleo func1 y el segundo asociado a la función del núcleo func2.

Por otra parte se observa que cada marco de página asociado a una función, tanto en la pila de usuario como en la pila del núcleo contiene la siguiente información: parámetros de la función, variables locales de la función, dirección de retorno después de la ejecución de la función (es una copia del contenido del registro contador del programa en el momento de creación del marco) y dirección de comienzo del marco de pila anterior (es una copia del contenido del registro puntero de la pila en el momento de creación del marco).

Tabla de procesos

La *tabla de procesos* es una estructura global del núcleo donde se almacena información de control relevante sobre cada proceso existente en el sistema. Cada entrada de la tabla de procesos contiene distintos campos con información sobre un determinado proceso, como por ejemplo:

- *El identificador del proceso (pid).*
- *Los identificadores de usuario (uid, euid) y de grupo (gid, egid) del proceso.*
- *Punteros que permiten al núcleo localizar la tabla de regiones por proceso y el área U del proceso.*
- *El estado del proceso.* Un proceso durante su tiempo de vida puede pasar por diferentes estados (se estudian en la sección 4.8), cada uno de los cuales posee ciertas características que determinan el comportamiento del proceso.
- *Punteros para enlazar a los procesos en diferentes listas que permiten al núcleo controlar a los procesos, como por ejemplo, la lista de procesos planificados para ejecución, la lista de procesos dormidos...*
 - *Canal o dirección de dormir asociada al evento por el que el proceso ha entrado en el estado dormido.*
- *Información asociada a la prioridad de planificación del proceso que es consultada por el algoritmo de planificación del núcleo para determinar que proceso debe pasar a utilizar el procesador.*
- *Información asociada al tratamiento de las señales como por ejemplo, las máscaras de las señales que son ignoradas, bloqueadas, notificadas y tratadas.*

- *Información genealógica*, que describe la relación de este proceso con otros procesos, como por ejemplo, el *pid* de su proceso padre, un puntero al primer hijo creado, un puntero al último hijo creado...
- *El tiempo de ejecución del proceso y el tiempo de utilización de los recursos de la máquina*. Estas informaciones son usadas por el núcleo, entre otras cosas, para el cálculo del valor de la prioridad de planificación del proceso.

Área U

El **área de usuario** o **área U** es una estructura local asociada a cada proceso que contiene información de control relevante sobre el mismo que el núcleo necesita consultar únicamente cuando ejecuta dicho proceso. Entre la información contenida en los campos del área U se encuentra:

- Un *puntero* a la entrada de la tabla de procesos asociada a dicho proceso.
- Los *identificadores de usuario* (*uid*, *euid*) y de grupo (*gid*, *egid*) del proceso. No debe extrañar la aparición de esta información tanto en el *área U* como en la entrada de la *tabla de procesos* asociada al proceso. Sin entrar en detalles más precisos, comentar únicamente que a los *identificadores de usuario y de grupo* almacenados en el *área U* pueden en determinadas circunstancias diferir de los almacenados en la tabla de procesos.
- Los argumentos de entrada, los valores de retorno y el identificador del error (si se produjese) de la llamada al sistema en ejecución.
- Las direcciones de los manipuladores de las señales y otras informaciones relacionadas.
- Información acerca de las regiones de código, datos y pila obtenida de la cabecera del programa.
- La *tabla de descriptores de ficheros* que mantiene información sobre los ficheros abiertos por el proceso.
- El *directorio de trabajo actual* y el *directorio raíz actual*.
- La *terminal de acceso* asociada con el proceso, si existe alguna.
- Estadísticas de uso la CPU.

El *área U* de un proceso se puede considerar en ciertos aspectos como una extensión de la entrada asignada a dicho proceso en la *tabla de procesos*. Sin embargo, mientras que la información contenida en la tabla de procesos debe ser estar siempre accesible para el núcleo, el *área U* contiene información que necesita únicamente estar accesible para el núcleo cuando se está ejecutando el proceso.

El núcleo puede acceder directamente a los campos del *área U* del proceso que se está ejecutando pero no al *área U* de otros procesos. Internamente, el núcleo referencia a una variable denominada *u* para acceder al *área U* del proceso (A) que actualmente se está ejecutando. Cuando se ejecuta otro proceso (B), el núcleo reorganiza su espacio de direcciones virtuales de forma que la variable *U* referencie al *área U* del nuevo proceso B. Esta implementación permite al

núcleo identificar fácilmente al proceso actual siguiendo el campo puntero del *área U* que apunta a la correspondiente entrada de la tabla de procesos.

Tabla de regiones por proceso

La **tabla de regiones por proceso** es una estructura local a cada proceso que contiene una entrada por cada región (código, datos y pila de usuario). Si existen regiones de memoria compartida cada una de ellas también tendrá asignada una entrada.

Cada entrada de la *tabla de regiones por proceso* apunta a una entrada en la *tabla de regiones* y contiene la dirección virtual de comienzo de una región. Este nivel extra de direccionamiento (desde la *tabla de regiones por proceso* a la *tabla de regiones*) permite que procesos independientes puedan compartir regiones. Además cada entrada contiene el tipo de acceso permitido al proceso sobre dicha región: sólo lectura, lectura y escritura, o lectura y ejecución.

La *tabla de regiones por proceso* de un proceso puede implementarse dentro de la entrada de la tabla de procesos asociado a dicho proceso, o puede implementarse dentro del *área U* de dicho proceso. También puede implementarse en una *área de memoria independiente*.

Tabla de regiones

La **tabla de regiones** es una estructura global del núcleo que contiene una entrada por cada región asignada por el núcleo. Cada entrada de esta tabla contiene la información necesaria para describir una región, como por ejemplo:

- Un puntero al nodo-i del fichero cuyo contenido fue cargado dentro de la región.
- El tipo de región (código, datos, pila de usuario o memoria compartida).
- El tamaño de la región.
- La localización de la región en memoria principal, típicamente un puntero a una *tabla de páginas*.
- El estado de la región, que puede ser una combinación de: bloqueada, bajo demanda, en proceso de ser cargada en memoria y válida (cargada en memoria).
- El contador de referencia, que indica el número de procesos que están referenciando a una región.

Las entradas de la tabla de regiones se organizan en dos listas: una *lista enlazada de regiones libres* y una *lista enlazada de regiones activas*. Simultáneamente una entrada sólo puede pertenecer a una de las dos listas

Ejemplo:

En la siguiente figura se representa un posible diagrama que relaciona a las estructuras de datos del núcleo asociadas a los procesos: pila del núcleo, área U, tabla de procesos, tabla de regiones por proceso y tabla de regiones. En concreto se han considerado dos procesos: el proceso A y el proceso B.

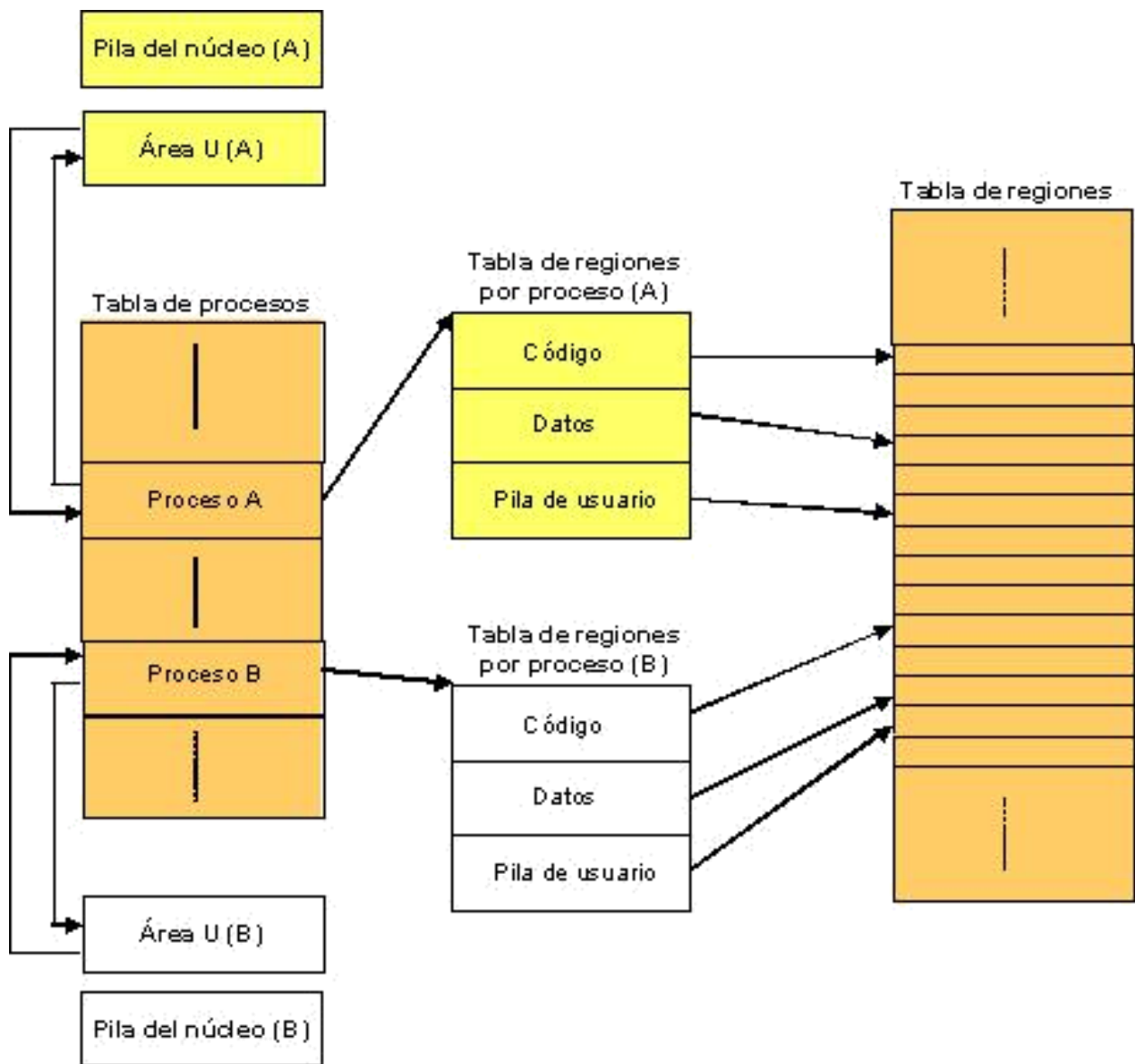


Figura. Estructura de datos del núcleo asociadas a los procesos A y B

En este diagrama se observa claramente el carácter local y global de las diferentes estructuras. Así el proceso A y el proceso B poseen cada uno de ellos su propia **pila del núcleo**, **área U** y **tabla de regiones por proceso**.

Mientras que la **tabla de procesos** y la **tabla de regiones** son comunes para todos los procesos.

Asimismo, en este diagrama se observa claramente la relación existente entre estas estructuras. Por ejemplo, el **área U** del proceso A apunta a la entrada asociada a dicho proceso en la **tabla de procesos**. A su vez dicha entrada posee un puntero tanto al **área U** como a la **tabla de regiones por proceso** asociadas al proceso A. Esta tabla posee tres entradas asociadas cada una de ellas a las regiones de código, datos y pila de usuario del proceso A. Cada una de las entradas de la **tabla de regiones por proceso** contiene un puntero a una entrada de la **tabla de regiones**. Relaciones análogas se aprecian para el proceso B.

Además, se observa en la tabla de regiones que el proceso A y el proceso B no tienen ninguna región común. Por lo tanto, el contador de referencias de estas regiones estará en 1, ya que sólo un proceso, el A o el B, está referenciando a través de una entrada de su tabla de regiones por proceso a cada región.